

Organização estrutural e arquitetura do projeto:

Visando otimizar o fluxo de trabalho e mitigar riscos de integração, a raiz do projeto foi dividida entre parte 1 e parte 2. Essa separação permitiu a atuação eficiente em subequipes isoladas, garantindo que o progresso de cada etapa acontecesse de forma simultânea sem a ocorrência de conflitos de mesclagem ou interferências mútuas no código-fonte. Seguindo este princípio de clareza arquitetural, a raiz da parte 2 foi estruturada em duas pastas independentes: back end e front end. Essa divisão serviu por conta da organização dos componentes de react, que tem um ecossistema caracterizado pela geração de múltiplos arquivos utilitários dispersos. Ao isolar a camada de apresentação da camada de serviços, estabeleceu-se um ecossistema limpo e de fácil manutenção. Dentro da pasta parte 2 também há o relatório da parte 2 de grafos e parte 2 de AVD.

No interior da pasta de back end, preservou-se a estrutura padrão originalmente fornecida pelo modelo da rubrica do projeto. Porém, introduziu-se a adição do arquivo `app.py`, um arquivo para implementar as rotas de comunicação via API REST, permitindo que a interface gráfica consuma dinamicamente as estruturas e as métricas computadas no servidor Flask.

Procedimentos de execução e testes:

Para rodar a interface de consultas customizadas e algoritmos individuais, utiliza-se o módulo de CLI fornecendo os argumentos de algoritmo, origem e destino desejados. Caso queira salvar o resultado em formato legível, basta incluir o argumento de saída correspondente: `python -m src.cli --algoritmo DIJKSTRA --origem "The Matrix" --destino "Avatar" --out ./out/` (se quiser substituir os filmes altere o conteúdo dentro das aspas)

Para a validação automatizada das regras de consistência lógica e cobertura de falhas conceituais, como a rejeição de pesos negativos pelo dijkstra e o bloqueio de ciclos infinitos pelo Bellman-Ford, a suite de testes foi segmentada de forma estrita. A execução de todos os cenários pode ser disparada diretamente pelo framework do pytest: `python -m pytest`

Por fim, o processamento analítico massivo e global do dataset de filmes, responsável por submeter a rede a dezenas de rotas aleatórias com o intuito de alimentar o painel estatístico e consolidar o arquivo unificado de benchmarking do dashboard visual, é acionado por meio do script de resolução estruturado: `python -m src.solve`

Contexto:

A definição dos pesos das arestas foi estabelecida com base no conceito de Frequência de colaboração/interseção de elenco. Uma aresta é criada entre dois filmes somente se eles compartilham pelo menos um ator em seu elenco principal. O peso dessa conexão é diretamente proporcional ao número de atores em comum entre os dois filmes. Se o filme X e o filme Y compartilham 3 atores na amostragem processada, a aresta $E(X,Y)$ recebe o peso estrito de 3.0.

Essa escolha de peso é crucial por dois motivos fundamentais:

- Métrica de proximidade temática e industrial: Pesos maiores indicam núcleos fortes de produção ou franquias consolidadas (como Star Trek), mapeando panelinhas de Hollywood.
- Inversão para algoritmos de caminho mínimo: Como o algoritmo de Dijkstra busca minimizar a soma dos pesos ao longo de um percurso, fomos confrontados com um desafio conceitual: na nossa modelagem, pesos maiores representam maior afinidade, mas o algoritmo interpreta pesos maiores como caminhos mais longos ou custosos. Para manter a coerência operacional, o custo foi interpretado como a resistência ao distanciamento ou caminhos de conexões densas, sendo assim um desafio interessante.

Para validar os limites teóricos do algoritmo de Bellman-Ford, que exige cenários com custos estritamente negativos, implementamos uma lógica artificial de injeção de pesos. Criamos mini-grafos isolados em memória simulando ciclos negativos. Adicionalmente, forçou-se um ciclo negativo fechado entre três produções fictícias para validar a capacidade do algoritmo de interromper sua execução e disparar o sinal de erro preventivo antes do estouro de memória.

Exploração:

Ao processar o dataset consolidado do TMDb no backend, a modelagem foi executada de forma irrestrita, computando todas as adjacências e interseções de elencos sem cortes prévios na base de dados principal. Sob essa perspectiva macro, o grafo alcançou uma ordem de 4.803 nós (filmes) e um tamanho massivo de 149.537 arestas (conexões).

Os algoritmos de varredura cega (BFS e DFS) aplicados sobre a matriz de adjacências completa demonstraram empiricamente a existência de um componente gigante altamente conectado, capaz de abraçar mais de 95% de toda a malha de produções através de poucos saltos relacionais. Contudo, essa densidade original impõe um desafio crítico para a camada de visualização de dados. Plotar 149 mil

arestas simultaneamente em uma interface web resultaria em um colapso de legibilidade e degradação de performance por saturação de pixels (hairball effect).

Para solucionar essa limitação e viabilizar uma experiência de usuário fluida e analítica, estabeleceu-se uma estratégia de dissociação entre o processamento e a exibição: enquanto os motores de cálculo do backend operam sobre a totalidade das 149.537 arestas para garantir a precisão matemática absoluta dos caminhos mínimos, a interface gráfica do frontend aplica uma filtragem seletiva, renderizando os elos baseados estritamente nos 5 atores principais de cada obra. Esse refinamento reduziu a carga visual na tela para 3.900 arestas, gerando uma malha limpa, expressiva e geometricamente interpretável.